

# 通行止め・通行困難地点 公開機能 セキュリティレビュー

**バージョン:** 1.2 **最終更新日:** 2026年7月28日 **対象範囲:** 2026年7月の公開API化(P2: Vercel Function + Blob)で追加・変更したコード（`api/closures.js`、`public/closures.js`、`public/config.js`、`public/service-worker.js`、`public/map.js`、`public/index.html`） **関連:** [設計書](#) `closures-design-202607.md` / [テスト計画](#) `closures-test-plan-202607.md`

**本アプリ（minoh-hiking）は表示専用であり、公開トークンを一切扱わない。** 公開UI（地点の編集・公開・トークン保存）は外部の運用アプリ（別リポジトリ・静的ホスティング）にある。**2～6章は、公開UI が本アプリにあった時点の評価を記録として残したもので、現在の本アプリの評価は 8章を参照すること。**

## 1. 前提とスコープ

- 本アプリは箕面エリア向けの小規模な公開PWA。今回の評価は**通行止め・通行困難地点の公開機能**が中心。
- \*\*扱うデータは「公開して良い安全情報」\*\*であり、個人情報・認証済みユーザー資産・決済等は扱わない。
- 攻撃者にとっての価値は低いが、**通行止め情報は登山者の安全判断に関わるため**、「偽情報の掲載・全データ消去」が最も影響の大きいシナリオになる。
- 書き込み口は `POST /api/closures` の**1本のみ**で、**共有トークン1個**で保護している。

### 深刻度の基準

| 深刻度 | 目安                             |
|-----|--------------------------------|
| 高   | 起こると影響が大きい（安全情報の改ざん・全消去 等）。要対応 |
| 中   | 条件付きで悪用され得る／多層防御を厚くすべき。対応推奨    |
| 低   | 影響限定的・発生条件が厳しい。把握しておけばよい       |

## 2. リスク一覧

| #  | 箇所                                              | 種別          | 深刻度        |
|----|-------------------------------------------------|-------------|------------|
| R1 | 共有トークン1個で全置換公開ができる                              | 権限設計・データ完全性 | 高（トークン漏洩時） |
| R2 | <code>POST</code> にレート制限がなく、トークン総当たりが可能         | 認証          | 中          |
| R3 | 公開トークンをブラウザ <code>localStorage</code> に保存       | 秘密情報の露出     | 中          |
| R4 | Leaflet 本体JSが <code>importmap</code> 経由で SRI なし | サプライチェーン    | 中          |

| #  | 箇所                                                        | 種別              | 深刻度     |
|----|-----------------------------------------------------------|-----------------|---------|
| R5 | リクエストボディのサイズ・件数の上限なし                                      | リソース枯渇（DoS/コスト） | 低～中     |
| R6 | CORS が <code>Access-Control-Allow-Origin: *</code>        | クロスオリジン         | 低（情報）   |
| R7 | <code>readStaticFallback</code> が Host ヘッダで fetch 先を組み立てる | SSRF            | 低       |
| R8 | トークン入力が <code>prompt()</code> （平文表示）                      | 覗き見・UX          | 低       |
| R9 | 誤操作で 0 件公開＝全消去                                            | 運用事故            | 低（警告あり） |

### 3. 各リスクの詳細と対策

#### R1【高】共有トークン1個で全置換公開ができる

- **内容:** `POST /api/closures` は正しいトークンさえあれば、送られた geojson で公開データを丸ごと置換する（`api/closures.js` の `handlePost`）。承認ステップや二人体制はなく、誰が公開したかの本人特定もできない（履歴スナップショットにデータは残るが操作者情報は残らない）。
- **影響:** トークンが漏れると、偽の通行止め情報の掲載や、0 件公開による全消去が可能。安全情報のため、「本当は通行止めなのに『通行止めなし』にされる」等は登山者のリスクにつながる。今回の設計で最も影響が大きい点。
- **条件:** トークンの漏洩・総当たり成功が前提（R2～R4 と連鎖）。
- **対策:**
  - トークンを十分長いランダム値にする（32文字以上のランダム推奨）。推測・辞書攻撃を実質不能にする。
  - 定期的なトークンローテーション（Vercel 環境変数を更新するだけ）。
  - 影響低減として、復旧手段の確保（履歴スナップショットは実装済み。R1 は予防、履歴は事後復旧）。
  - 将来的に厳格化するなら、公開の\*\*監査ログ（操作者・IP・日時）\*\*の付与を検討。

#### R2【中】POST にレート制限がなく、トークン総当たりが可能

- **内容:** `POST` はトークン照合のみで、試行回数の制限がない。エンドポイントは公開URLなので、攻撃者は連続でトークンを試せる。
- **影響:** トークンが短い・単純だと総当たりで破られ、R1 に直結する。
- **対策:**
  - Vercel Firewall / レート制限を該当パスに設定する（プラットフォーム側で最も手軽）。
  - トークン照合自体はタイミングセーフ比較を実装済み（`tokenEquals`、SHA-256 + `timingSafeEqual`）なので、タイミング攻撃には耐性あり。残る主対策はレート制限とトークン強度。

#### R3【中】公開トークンをブラウザ localStorage に保存

- **内容:** 初回入力後、トークンを `localStorage`（キー `minoh-hiking.closure-publish-token`）に保存する（`config.js` / `app.js` の `publishClosureData`）。
- **影響:** 同一オリジンで動く任意のJS（XSS や R4 の汚染CDN）から読み取れる。読み取られると R1 に直結。端末の物理アクセスでも取り出し得る。

- **対策:**
  - アプリのポップアップは `escapeHtml` で **XSS 対策済み** (`map.js`)。この防御を今後も維持する (HTML を innerHTML で直接組む変更を避ける)。
  - R4 (SRI) を塞ぐことで、CDN 汚染経由の localStorage 窃取を防ぐ。
  - **運用端末を限定**し、共用PC・共用ブラウザで公開しない運用ルールにする。
  - より厳格にするなら「毎回入力 (保存しない)」オプションも選択肢 (利便性とのトレードオフ)。

#### R4 【中】Leaflet 本体JSが importmap 経由で SRI なし

- **内容:** Leaflet の **CSS は SRI 付き**でピン留め済み (`index.html:13-15`) だが、**本体の ESM** (`leaflet-src.esm.js`) は **importmap** で読み込んでおり、**importmap は SRI (integrity) に非対応**のため、**JS には完全性チェックがない** (`index.html:18-21`)。
- **影響:** 万一 unpkg (CDN) が汚染されると、**アプリのオリジンで任意コードが実行**され、R3 のトークン窃取や公開データ改ざんに至り得る。発生確率は低いが、トークンが端末に載る今は影響が上がっている。
- **対策:**
  - **Leaflet をローカルに同梱** (`public/` に置いて自オリジンから配信) し、CDN 依存を外す。SRI 相当の完全性が得られ、オフラインPWAとの相性も良い (現状 SW で CDN をキャッシュしているが、初回汚染は防げない)。
  - もしくは importmap をやめ、`<script> + integrity` で読み込める構成に変更。
  - 少なくとも **バージョンピン留めは維持** (@1.9.4)。

#### R5 【低～中】リクエストボディのサイズ・件数の上限なし

- **内容:** `validateClosureGeoJSON` は形式・座標・id 重複は見るが、**features 件数やボディサイズの上限がない** (`api/closures.js`)。ボディ解析はプラットフォームが認証前に行う。
- **影響:** 認証済みトークン所持者による **巨大データ保存 (Blob 容量・転送コスト増)**、未認証でも Vercel の上限 (約4.5MB) までの解析負荷。小規模用途では実害は限定的。
- **対策:**
  - `data.features.length` に **上限** (例: 数百件) を設けて超過は **400**。
  - 必要なら `vercel.json` の Function 設定でボディ上限を絞る。

#### R6 【低・情報】CORS が `Access-Control-Allow-Origin: *`

- **内容:** GitHub Pages 版から Vercel の API を叩くために `*` を許可 (`api/closures.js` の handler)。
- **評価:** **GET は公開データなので妥当**。**POST はトークンが必須**で、クロスオリジンのJSは **他オリジンの localStorage を読めない**ため、`*` でもトークンは盗めない (= CORS 単体での悪用は成立しない)。
- **対策:** 現状維持で可。厳格化したいなら、POST 応答の許可オリジンを本番ドメインに限定することは可能 (GET は `*` のまま)。

#### R7 【解消済み】readStaticFallback が Host ヘッダで fetch 先を組み立てる

- **内容:** Blob 未作成時、`x-forwarded-host / host` から `https://{host}/data/minoh-hiking-closure.geojson` を取得していた (旧 `readStaticFallback`)。
- **影響:** もしヘッダ注入が可能な経路があれば、**固定パスの GET を別ホストに向ける軽微な SSRF**になり得た。

- **対応 (2026-07-18)** : 静的フォールバック自体を廃止し、readStaticFallback と同梱ファイル public/data/minoh-hiking-closure.geojson を削除した。**本リスクは解消** (GET は Blob 取得のみ)。

R8【低】トークン入力に prompt() (平文表示)

- **内容**: 初回のトークン入力に prompt() を使用 (app.js)。入力文字が画面に見える。
- **影響**: 肩越しの覗き見・画面共有時の露出。
- **対策**: 影響は小さい。厳格にするならマスク付き入力UIに置換。運用上は「人前で公開操作をしない」で足りる。

R9【低】誤操作で 0 件公開＝全消去

- **内容**: 全置換方式のため、0 件の geojson を公開すると全地点が消える。
- **緩和済み**: 0 件時は専用の警告付き確認ダイアログを表示する実装済み (app.js の publishClosureData)。
- **対策**: 現状の警告で概ね十分。心配なら「直前バージョンへ戻す」導線 (履歴スナップショットの活用) を将来追加。

4. すでに守れている点 (評価できる実装)

- **サーバー側での多層バリデーション** (形式・Point・座標範囲・id 重複・version 必須) をクライアントと二重に実施。
- **タイミングセーフなトークン比較** (SHA-256 + timingSafeEqual)。
- **トークン未設定時は 503 で fail-closed** (誤って全開放されない)。
- **ポップアップの XSS 対策** (escapeHtml で & < > " ' をエスケープ)。
- **トークンをコードに埋め込まない運用** (Vercel 環境変数)。.gitignore が .env\* を除外。
- **履歴スナップショット** を Blob に保存し、事後復旧の余地を確保。
- **Leaflet CSS は SRI 付き** でバージョンピン留め。
- **認証チェックをバリデーションより前に実施** (未認証者に検証詳細を返さない)。

5. 推奨対応の優先順位

| 優先 | 対応                                   | 対応するリスク | 手間         |
|----|--------------------------------------|---------|------------|
| 1  | トークンを長いランダム値にする+定期ローテーション            | R1, R2  | 小 (環境変数のみ) |
| 2  | POST パスにレート制限 (Vercel Firewall)      | R2      | 小〜中        |
| 3  | Leaflet をローカル同梱し CDN 依存を外す (またはSRI化) | R4, R3  | 中          |
| 4  | features 件数の上限チェックを追加                | R5      | 小          |
| 5  | 運用端末の限定・共用環境での公開禁止 (運用ルール)           | R3, R8  | 小          |
| 6  | フォールバック先ホストの固定                       | R7      | 小          |

最優先は 1・2。R1（全置換の影響の大きさ）は設計上避けられないため、「トークンを破られないこと」に対策を集中させるのが費用対効果が高い。

## 6. 残留リスク・対象外

- 公開データ（GET）は誰でも取得可能＝設計どおり（安全情報のため許容、設計書 §4.4）。
- 本人特定つき監査ログ・多人数の同時編集制御は今回対象外（設計書 §12）。厳密な追跡が必要になれば別途設計。
- 端末の物理的な乗っ取り（ロック解除済み端末を奪われる等）は本レビューの対象外（端末管理の領域）。
- 本レビューは今回追加・変更したコードが対象。既存のタイルDL・移動記録等は範囲外。

## 8. 現構成での評価（表示専用化後）

公開UI（ファイル読み込み・マップに反映・公開・トークン保存）が本アプリから無くなった。書き込み口（POST /api/closures）とトークン方式そのものは変えていないため、R1・R2・R5・R6 の評価は変わらない（対策の優先順位も 5章のまま）。変化したのは次の点である。

### 8.1 本アプリ（minoh-hiking）で解消したもの

| #  | 変化                                                                                                                                                      |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| R3 | 解消。公開トークンを扱わなくなった。旧キー minoh-hiking.closure-publish-token は closures.js の冒頭で localStorage.removeItem() して消している（次のリリースでこの後始末コードも撤去）。リスク自体は公開UI側へ移動（→ 8.2） |
| R4 | 影響度が低下。CDN 汚染で任意コードが走っても、本アプリのオリジンにはもう公開トークンが無い（SRI 化の推奨自体は据え置き）                                                                                        |
| R8 | 消滅（prompt() によるトークン入力が無くなった）                                                                                                                            |
| R9 | 0 件公開の警告確認は公開UI側の責務。本アプリからは公開操作自体ができないため、誤操作の入口が1つ減った                                                                                                   |

### 8.2 公開UI側（外部の運用アプリ）で評価すべき点

公開UI は静的ホスティング（GitHub Pages）に置かれる。本レビューの範囲外だが、リスクがそのまま移るため引き継ぎ事項として記録する。

| #  | 内容                | 深刻度 | 評価・対策                                                                                                                                                          |
|----|-------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| N1 | 公開トークンの保存オリジンが変わる | 中   | R3 と同質のリスクがオリジンごと移動する。GitHub Pages のユーザーサイトは同一アカウントの他アプリとオリジンを共有するため、同一オリジンの別アプリに XSS があればトークンを読める。運用端末の限定（共用環境で公開しない）を引き続きルールとし、トークンの定期ローテーション（5章の優先1）をより重視する |

| #  | 内容                             | 深刻度 | 評価・対策                                                                                                                                                                                                                                                                                          |
|----|--------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| N2 | 静的ホスティングではCSP等のレスポンスヘッダを制御できない | 中   | 任意のレスポンスヘッダを付けられず、 <code>Content-Security-Policy</code> ・ <code>X-Frame-Options</code> 等をサーバー側で強制できない。 <code>&lt;meta http-equiv="Content-Security-Policy"&gt;</code> で代替できる範囲に限られる( <code>frame-ancestors</code> はmetaでは効かない)。外部CDNに依存せず自オリジンに同梱する(R4の対策と同じ方針)ことで、meta CSPを厳しく書ける構成にしておくのが現実的 |
| N3 | 公開前の実機プレビューが無く、誤りが一時的にユーザーへ出る  | 低～中 | 設計上の受容事項。全置換+履歴スナップショットで復旧可能(→ <a href="#">設計書 §7</a> )                                                                                                                                                                                                                                        |
| N4 | 検証ルールを公開UI側に持たない(サーバー任せ)       | 低   | 意図した設計(→ <a href="#">設計書 §5.5</a> )。同じルールを両側に持つとズレるため、判定はサーバーに一本化する。セキュリティ境界はサーバー側にあり、この分離は妥当                                                                                                                                                                                                  |

## 9. 変更履歴

| 日付         | バージョン | 内容                                                                                                                              |
|------------|-------|---------------------------------------------------------------------------------------------------------------------------------|
| 2026-07-16 | 1.0   | 初版(公開API方式のセキュリティレビュー)                                                                                                          |
| 2026-07-18 | 1.1   | R7(Hostヘッダ由来のSSRF懸念)を <b>解消済み</b> に更新(静的フォールバックと <code>readStaticFallback</code> の削除による)。あわせて旧git公開スクリプト(bat/ps1)の削除を反映         |
| 2026-07-28 | 1.2   | 表示専用化を反映。8章を追加し、本アプリで解消したもの(R3・R8・R9、R4は影響低下)と、公開UI側で評価すべき点(N1 トークン保存オリジン、N2 静的ホスティングでのCSP制御の弱さ、N3 実機プレビューの不在、N4 検証のサーバー一本化)を整理 |